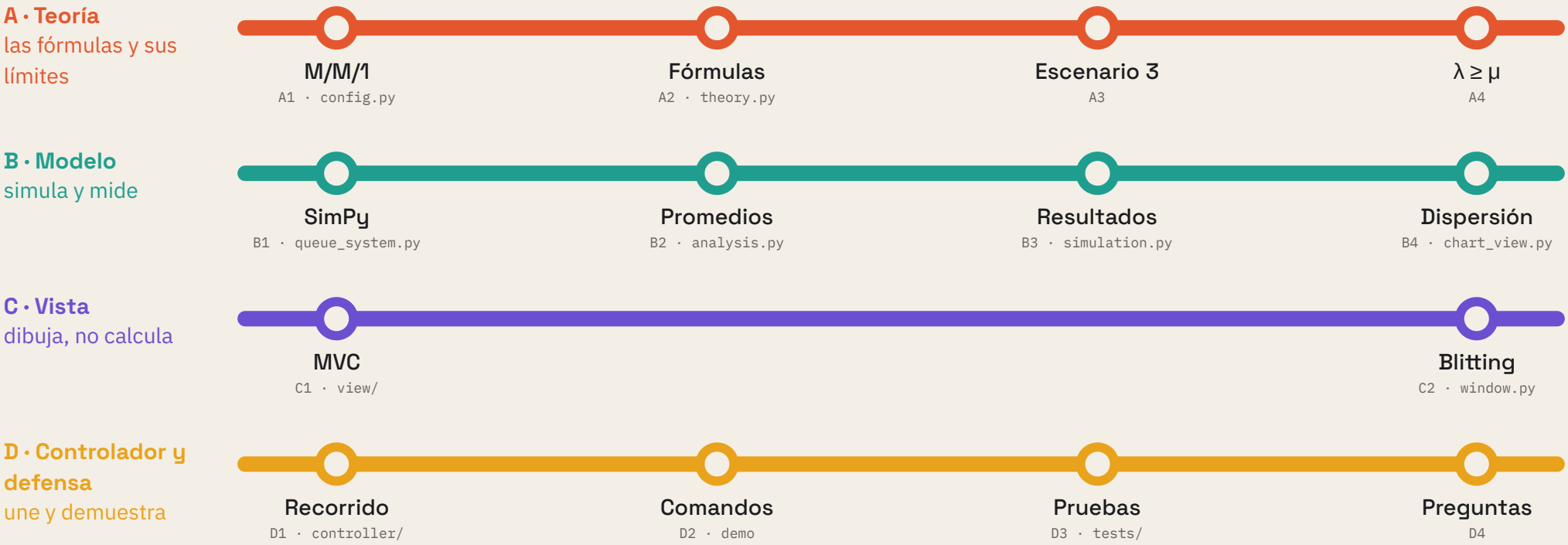
La caja rápida como cola M/M/1

Qué hace cada archivo del código, de dónde sale cada número de la tabla y cómo responder lo que pueda preguntar el docente. Semilla 42, 20 000 minutos simulados, λ = 2.0 · 2.5 · 2.9 y μ = 3.0 clientes/min.

Cómo usarla. El mapa enlaza cada ficha. Las preguntas enlazan su respuesta. TIP marca lo que no sale de las diapositivas de la semana 3.

ÍNDICE · CADA COLOR ES UNA CAPA

Mapa del código



TEORÍA · MODEL/CONFIG.PY

A1 Qué es un M/M/1

M Llegadas de Poisson

Llegan en promedio λ clientes por minuto. El tiempo entre dos llegadas es exponencial con media 1/λ.

> **Imaginalo:** en la caja rápida entran 2 clientes por minuto, pero nunca a intervalos fijos.

Ojo · λ es una tasa (clientes/min); 1/λ es un tiempo (min).

M Servicio exponencial

La cajera atiende a μ clientes por minuto cuando está ocupada; cada cobro dura en promedio 1/μ minutos.

> **Imaginalo:** μ = 3 significa que un cobro dura 20 segundos en promedio.

Ojo · "Exponencial" no es "constante": hay cobros muy cortos y unos pocos muy largos.

1 Un servidor FIFO

Una sola caja y una fila única: se atiende en orden de llegada. La utilización es ρ = λ/μ.

> **Imaginalo:** ρ = 0.667 → la caja está ocupada el 66.7 % del tiempo.

Ojo · La guía escribe ρ invertido (μ/λ). Lo correcto es λ/μ, como en el pseudocódigo de la guía.

En el código: **Scenario** guarda (λ, μ) y calcula **utilization**; **SCENARIOS** son los tres de la guía; **SimulationConfig** fija 20 000 min y semilla 42. Todo en minutos.

A2

Fórmulas del estado estable

ρ Utilización

$$\rho = \frac{\lambda}{\mu}$$

Fracción del tiempo con la caja ocupada.

L Clientes en el sistema

$$L = \frac{\lambda}{\mu - \lambda} = \frac{\rho}{1 - \rho}$$

En la fila más el que está pagando.

Lq Clientes en la cola

$$L_q = \frac{\lambda^2}{\mu(\mu - \lambda)} = L - \rho$$

Solo los que esperan.

W Tiempo en el sistema

$$W = \frac{1}{\mu - \lambda}$$

Desde que llega hasta que se va.

Wq Espera en cola

$$W_q = \frac{\lambda}{\mu(\mu - \lambda)} = W - \frac{1}{\mu}$$

Desde que llega hasta que lo atienden.

= Ley de Little

$$L = \lambda W \qquad L_q = \lambda W_q$$

Une clientes con tiempos: comprobá E1: 2 = 2·1.

Todas valen **solo si λ < μ**. Si no, **theoretical_metrics** lanza **UnstableSystemError** antes de simular. Las cinco viajan juntas en **QueueMetrics**, que también guarda las simuladas.

D1

CONTROLADOR · CONTROLLER/SIMULATION_CONTROLLER.PY

Recorrido de una ejecución

1 → 2 → 3 → 4 → 5 → 6

1 · __main__.py

Solo llama a main(). Si se corre por ruta, agrega codigo/ al path.

2 · parse_args

Lee --escenario, --llegada, --semilla... y rechaza combinaciones inválidas.

3 · report_scenarios

Por cada escenario estable llama a run_scenario; avisa los inestables.

4 · run_scenario

Teoría primero, luego simulate_mm1 y simulated_metrics.

5 · Vistas

console_view imprime la tabla; chart_view guarda 5 PNG.

6 · LiveWindow

Abre la ventana en vivo (salvo --sin-animacion).

Amarillo: controlador. Verde: modelo. Violeta: vista. El controlador es el único que importa a los dos lados: el modelo no sabe que existe una vista y la vista nunca llama a SimPy.

B1

El corazón SimPy

```
arrival_rng, service_rng = default_rng(seed).spawn(2)
env = simpy.Environment()
cashier = simpy.Resource(env, capacity=1)

def customer(record):
    with cashier.request() as request:
        yield request              # espera su turno
        record.service_start = env.now
        yield env.timeout(service_rng.exponential(1/μ))
        record.departure = env.now

def arrivals():
    while True:
        yield env.timeout(arrival_rng.exponential(1/λ))
        customers.append(CustomerRecord(id, env.now))
        env.process(customer(record))

env.process(arrivals()); env.run(until=sim_time)
```

Qué tenés que saber decir

Environment es el reloj y la agenda de eventos (**env.now** = minuto actual). **Resource(capacity=1)** es la caja: un cliente a la vez, el resto en su cola FIFO.

yield pausa el proceso hasta que ocurre el evento: **request** espera la caja, **timeout(t)** deja pasar t minutos simulados.

exponential(1/tasa): NumPy pide la media (escala), no la tasa. Por eso 1/λ y 1/μ.

spawn(2): dos flujos aleatorios independientes, uno para llegadas y otro para servicios, que salen de la misma semilla.

B2 · cómo se mide

B2

Promedio en el tiempo vs. por cliente

En el tiempo: L, Lq, ρ

occupancy_trace arma una escalera: llegar suma 1 al sistema y a la cola, pasar a la caja resta 1 de la cola, irse resta 1 del sistema.

time_average = área bajo la escalera ÷ tiempo total. ρ = tiempo con la caja ocupada ÷ tiempo total.

`pondera cada valor por cuánto duró`

vs

Por cliente: W, Wq

Cada **CustomerRecord** guarda arrival, service_start y departure. Wq de un cliente = service_start − arrival; W = departure − arrival.

Se promedia sumando y dividiendo entre cuántos clientes ya pasaron a la caja (Wq) o ya se fueron (W).

`cada cliente pesa lo mismo`

> **Si te preguntan "¿por qué no promediaste la cola cliente por cliente?": porque L es un promedio en el tiempo. 3 clientes durante 10 min pesan más que 3 clientes durante 1 s.**

MODELO · MODEL/SIMULATION.PY · RUN_SCENARIO

Resultados: semilla 42, 20 000 min

```
$ python -m simulacion_ape --sin-animacion
Métrica     E1 teo   E1 sim   E2 teo   E2 sim   E3 teo   E3 sim
ρ            0.667    0.670    0.833    0.835    0.967    0.968
L            2.000    2.036    5.000    5.029   29.000   26.592
Lq           1.333    1.366    4.167    4.193   28.033   25.624
W  (min)     1.000    1.022    2.000    2.016   10.000    9.183
Wq (min)     0.667    0.685    1.667    1.681    9.667    8.849
Error       0.5–2.8 %         0.2–0.9 %         0.2–8.6 %
Clientes        39 860            49 888            57 931
```

Cómo leer la tabla

E1 y E2 quedan a menos de 3 % de la teoría: la simulación está bien construida.

ρ casi no tiene error en ningún escenario; L, Lq, W y Wq se disparan al acercarse a ρ = 1: con λ = 2.9 la cola teórica es de 28 clientes.

Los clientes simulados ≈ λ · 20 000 (2·20 000 ≈ 39 860): otra comprobación rápida.

A3 · por qué E3 se aleja

A3

TEORÍA · INTERPRETACIÓN

Por qué el escenario 3 se aleja ~8 %

1 No es un bug

ρ simulado = 0.968 contra 0.967 teórico: la caja trabaja lo que tiene que trabajar. Lo que cuesta estimar es el tamaño de la cola.

Ojo · No digas "la simulación está mal": E1 y E2 confirman que el modelo es correcto.

2 Cola muy correlacionada

Con ρ = 0.967 la caja casi nunca se vacía: una cola larga dura mucho y los minutos seguidos se parecen. 20 000 min equivalen a pocas "historias" independientes. TIP

> **Imaginalo:** medir el clima de un año mirando solo una semana de lluvia.

3 Cómo se reduce

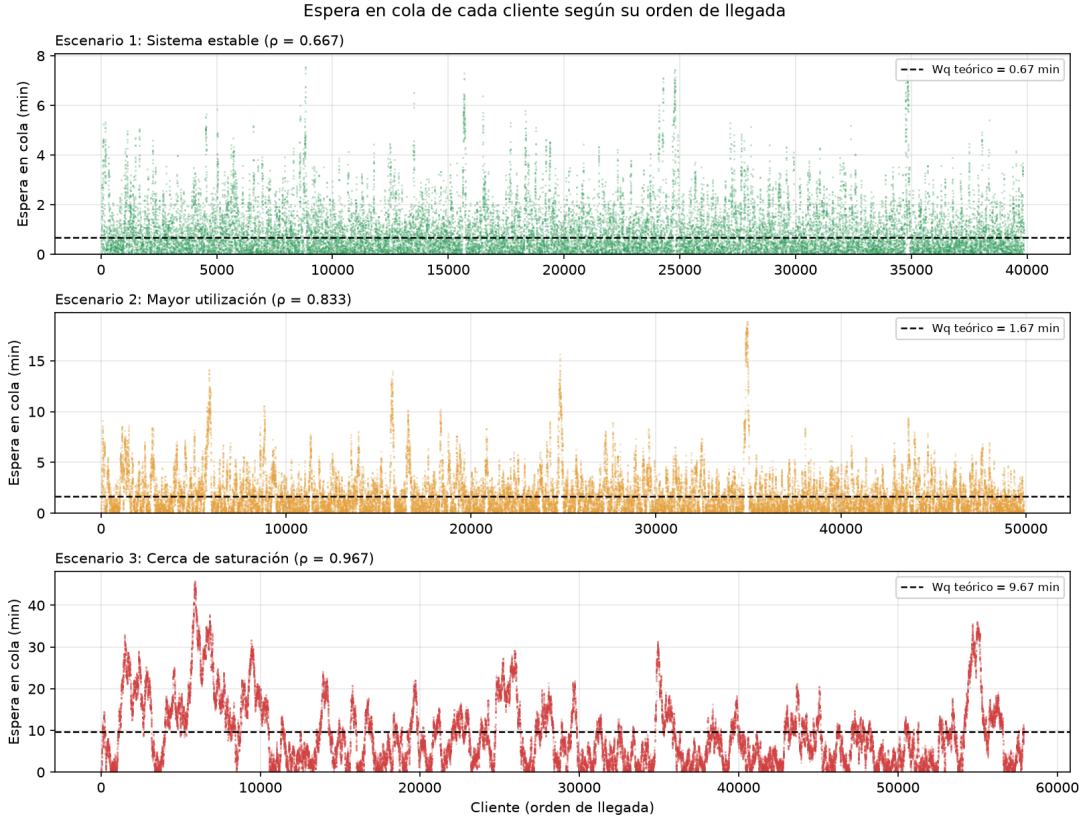
Más minutos simulados (--tiempo) o varias réplicas con semillas distintas y promediar. Con 100 000 min (semilla 42): L = 30.3, error 4 %. TIP

Ojo · Con semilla 1, L = 31.6: queda por encima de 29. El error cambia de signo, no es un sesgo fijo.

Frase para la defensa: "Cerca de la saturación el sistema es muy sensible: pequeñas subidas de λ disparan la cola, y la simulación necesita mucho más tiempo para converger a la teoría".

B4

Diagrama de dispersión de la espera



1 Cómo se lee

Cada punto es un cliente: x = su orden de llegada, y = minutos que esperó en la cola. La línea punteada es W_q teórico. Cada panel tiene su escala.

2 Qué muestra

E1: máximo 7.7 min, picos aislados. E2: máximo 18.8 min. E3: rachas que llegan a 45.8 min; hasta 2 636 clientes seguidos esperan más que W_q (en E1, solo 124).

3 Por qué importa

Un cliente que espera mucho deja esperando mucho al siguiente. Esas rachas son la razón por la que E3 converge lento y se aleja ~8 %.

Ojo · No lo pide la guía: lo agregamos porque el docente mostró dispersión en clase y la guía pide "gráficas de los tiempos de espera".

C1 ARQUITECTURA · MODEL/ · VIEW/ · CONTROLLER/

MVC: quién hace qué

M model/

config parámetros · **theory** fórmulas · **queue_system** SimPy · **analysis** métricas simuladas · **simulation** une teoría y simulación · **state** foto del sistema en un minuto · **live** corrida lista para la ventana.

Ojo · El modelo no importa Matplotlib ni la consola.

V view/

console_view tabla · **chart_view** 5 PNG del informe · **live/window** ventana · **live/scene** el local animado · **live/board** tablero · **live/charts** 3 gráficas · **live/theme** colores.

Ojo · La vista solo dibuja lo que recibe; no calcula métricas.

C controller/

Lee argumentos, pide corridas al modelo y entrega los resultados a las vistas. Le pasa a la ventana el callback **make_run(λ, μ)**.

> **Imaginalo:** un mozo: toma el pedido, lo lleva a la cocina y trae el plato.

Ojo · **lru_cache** en make_run: volver a un escenario no vuelve a simular.

Por qué el callback: la ventana necesita simular cada vez que movés λ o μ, pero si importara el modelo directamente se rompería la separación. Con **make_run** la ventana solo conoce "dame una corrida para (λ, μ)".

C2

Ventana en vivo y blitting

1 El problema

Redibujar toda la figura (~150 textos) cuesta ~200 ms: la animación se trababa.

2 Fondo guardado

Tras un dibujo completo, **copy_from_bbox** guarda una foto de lo fijo: ejes, títulos, curvas teóricas.

3 Dos capas

fast: clientes, reloj, contadores, cada cuadro. **slow**: histogramas, columna viva y avisos, cada 0.25 s.

4 Cada cuadro

restore_region pega el fondo, **draw_artist** pinta solo lo que se mueve, **blit** lo copia a la pantalla. ~14 ms.

Quién marca el ritmo

Un temporizador del canvas (**new_timer**, cada 10 ms) llama a **_on_frame**, que avanza el reloj según el tiempo real transcurrido y la velocidad elegida.

Ojo · No es FuncAnimation, aunque el README de código lo diga: es temporizador + blitting manual.

La ventana no simula

Cada cuadro le pregunta a **LiveRun** el estado (**state_at**) y las métricas (**metrics_at**) en el minuto actual. Al mover λ o μ pide otra corrida con la misma semilla.

A4

Sistema inestable: λ ≥ μ

? Qué significa

Llegan tantos o más clientes de los que la caja puede atender: la cola crece sin límite y no existe estado estable.

> **Imaginalo:** μ − λ ≤ 0 deja las fórmulas con división por cero o negativas.

⌨ En la consola

report_scenarios pregunta **is_stable** antes de simular. Si da falso, **show_unstable** avisa y ese escenario no se simula ni entra en la tabla.

Ojo · Se valida antes de simular, como dice el pseudocódigo de la guía.

▶ En la ventana

run_live simula solo los minutos que se muestran. La animación sigue (la cola crece), aparece "Sistema inestable: λ ≥ μ, la teoría no aplica" y se ocultan teoría y corrida larga.

D2

DEMO · DESDE LA CARPETA CODIGO/

Comandos para la demo

```
$ python -m venv .venv
$ .venv/bin/pip install -r requirements.txt
$ .venv/bin/python -m simulacion_ape
  tabla + 5 PNG en resultados/ + ventana
$ .venv/bin/python -m simulacion_ape --escenario 3
$ .venv/bin/python -m pytest
  24 passed
```

Si el docente pregunta "¿cómo lo instalo?": solo **requirements.txt**, con versiones fijadas.

```
$ python -m simulacion_ape --llegada 1.5 --servicio 2
  λ y μ propios (van juntos)
$ python -m simulacion_ape --semilla 7
  otros números, mismas conclusiones
$ python -m simulacion_ape --tiempo 100000
  E3 se acerca más a la teoría
$ python -m simulacion_ape --sin-animacion
```

En la ventana: **espacio** pausa, **R** reinicia, ← → velocidad, **1 2 3** escenarios; deslizadores λ y μ.

PRUEBAS · CODIGO/TESTS/ · PYTEST

Qué prueban los 24 tests

M test_model.py

Fórmulas del escenario 1; λ ≥ μ se rechaza; la misma semilla repite los clientes; la caja atiende en orden FIFO; la simulación converge a la teoría; L se pondera por tiempo; el estado en un minuto no ve el futuro.

C test_app.py · test_charts.py

test_charts: save_charts crea dispersion_espera.png. **test_app**: El programa corre ejecutado por su ruta; la tabla mantiene los números de referencia; un escenario personalizado inestable no se simula; sin pantalla explica cómo habilitar la ventana.

V test_live_view.py

Sin abrir ventanas (backend Agg): los deslizadores vuelven a simular con la misma semilla; los botones de escenario; el tablero muestra lo mismo que el modelo; la pausa congela el reloj; cada cuadro solo repinta lo animado.

Por qué importa la semilla para los tests: con semilla fija la simulación es reproducible, así que se puede comparar contra números exactos.

AUTOEXAMEN · RESPONDÉ ANTES DE MIRAR

D4

Preguntas probables

01 ¿Por qué ρ = λ/μ y no μ/λ?

02 ¿Qué pasa si λ ≥ μ?

03 ¿Qué dice la ley de Little y cómo la comprobás?

04 ¿Por qué el escenario 3 tiene más error?

05 ¿Qué hace yield en un proceso de SimPy?

06 ¿Cómo calculan L simulado?

07 ¿Para qué sirve la semilla?

08 ¿Por qué exponential(1/λ) y no exponential(λ)?

09 ¿Dónde está el MVC y por qué la ventana no simula?

10 ¿Qué es el blitting y por qué lo usaron?

11 ¿Qué cambiarías para bajar el error?

12 ¿Cuál es el sistema real y qué supuestos hacen?

D4

Respuestas

01 Es la fracción del tiempo ocupada: lo que llega dividido entre lo que se puede atender. La guía lo escribe invertido; su pseudocódigo usa λ/μ. volver

02 La cola crece sin límite y no hay estado estable. La consola no lo simula; la ventana lo anima sin teoría. volver

03 L = λW y Lq = λWq. E1: L = 2 y λW = 2·1 = 2; simulado 2.036 ≈ 2·1.022. volver

04 Con ρ = 0.967 la cola está muy correlacionada y converge lento; 20 000 min no alcanzan. ρ sí coincide. volver

05 Pausa el proceso hasta que ocurre el evento: que la caja quede libre (request) o pasen t minutos (timeout). volver

06 Área bajo la escalera de clientes en el sistema ÷ tiempo total (occupancy_trace + time_average). volver

07 Hace la corrida reproducible: mismos números, mismos resultados, y los tests pueden comparar valores exactos. Se cambia con --semilla. volver

08 NumPy recibe la media (escala), no la tasa. La media entre llegadas es 1/λ y la del cobro 1/μ. volver

09 model/ calcula, view/ dibuja, controller/ conecta. La ventana recibe el callback make_run y nunca importa SimPy. volver

10 Guardar el fondo y repintar solo lo que se mueve: ~14 ms por cuadro en vez de ~200 ms. volver

11 Simular más tiempo (--tiempo) o varias réplicas con semillas distintas y promediar. `TIP` volver

12 Caja rápida de supermercado: llegadas de Poisson, servicio exponencial, una caja, fila FIFO y capacidad infinita. volver

CIERRE · SI SOLO TENÉS UN MINUTO

Lo que tenés que poder decir

1. Modelamos una caja rápida como M/M/1: llegadas de Poisson, cobro exponencial, una caja FIFO.
2. SimPy simula cada cliente; medimos L y Lq en el tiempo, y W y Wq por cliente.
3. E1 y E2 quedan a menos de 3 % de la teoría; E3, cerca de saturación, converge más lento.
4. El código es MVC: el modelo calcula, la vista dibuja y el controlador los conecta.
5. Semilla fija y 24 tests: los resultados se pueden repetir y verificar.